

```

/*
=====
=====
    app.js – Main Application Logic
    -----
    -----
    Orchestrates the entire application:
    - Fetch data from YouTube API on load
    - Render all chart sections dynamically
    - Handle search, filtering, theme
toggle, navigation
    - Generate all other pages (top100.html,
artists.html, etc.)

=====
===== */

window.HVC = window.HVC || {};

/**
 * Main Application Manager
 */
window.HVC.App = (() => {
    let allSongs = [];
    let rankedSongs = [];
    let currentFilter = "today";

    /**
     * Initialize the app on page load
     */

```

```

    async function init() {
        attachEventListeners();

        // -----
        -----
        // MODE 1: Firestore (recommended once
        /admin is set up) – songs,
        // artists and news are managed in the
        admin dashboard and stream
        // to the public site in real time. No
        YouTube API key needed here.
        // -----
        -----
        if (window.HVC.CONFIG.USE_FIRESTORE &&
        window.HVC.FirestoreSync) {
            try {
                window.HVC.FirestoreSync.start(({
                songs }) => {
                    allSongs = songs;
                    rankedSongs =
                window.HVC.Ranking.rankSongs(songs);
                    updateLastUpdated();
                    renderHomepage();
                    hideLoadingScreen();
                });
            } catch (error) {
                handleError(error);
            }
            return;
        }

        // -----
        -----
        // MODE 2: Direct YouTube fetch

```

```

(original behavior) – used only if
  // USE_FIRESTORE is false in
channels.js, e.g. before setting up
  // Firebase/admin at all.
  // -----
-----
  try {
    if (window.HVC.CONFIG.API_KEY ===
"YOUR_YOUTUBE_API_KEY_HERE") {
      showError(
        "API Key Not Configured",
        'Please replace
"YOUR_YOUTUBE_API_KEY_HERE" in channels.js
with your actual YouTube Data API key, or
set CONFIG.USE_FIRESTORE = true once your
admin panel has songs in it. See README.md
for instructions.'
      );
      return;
    }

    await fetchAndRankSongs();
    renderHomepage();
    setTimeout(() => hideLoadingScreen(),
300);
  } catch (error) {
    handleError(error);
  }
}

/**
 * Fetch all songs from YouTube and rank
them
 */

```

```

    async function fetchAndRankSongs() {
        const { songs, errors } = await
window.HVC.API.fetchAllSongs();

        if (songs.length === 0) {
            throw {
                type: 'NO_SONGS_FETCHED',
                message: 'No songs were found.
Check your API key and channel IDs.'
            };
        }

        allSongs = songs;
        rankedSongs =
window.HVC.Ranking.rankSongs(songs);

        // Log any channel errors but don't
fail
        if (errors.length > 0) {
            console.warn("Some channels failed to
load:", errors);
        }

        // Update last-updated time
        updateLastUpdated();
    }

    /**
     * Update the "last updated" timestamp in
filter bar
     */
    function updateLastUpdated() {
        const el =
document.getElementById("last-updated");

```

```

        if (el) {
            el.textContent = `Updated
${window.HVC.Ranking.getRelativeTime(new
Date())}`;
        }
    }

    /**
     * Hide the loading screen
     */
    function hideLoadingScreen() {
        const screen =
document.getElementById("loading-screen");
        if (screen) {
            screen.classList.add("is-hidden");
        }
    }

    /**
     * Show an error state banner
     */
    function showError(title, message) {
        hideLoadingScreen();
        const banner =
document.getElementById("state-banner");
        if (banner) {
            banner.innerHTML = `
                <h3>${escapeHtml(title)}</h3>
                <p>${escapeHtml(message)}</p>
                <button class="btn btn-gold"
onclick="location.reload()">
                    <svg viewBox="0 0 24 24"><path
d="M1 4v6h6M23 20v-6h-6"
stroke="currentColor" stroke-width="2"

```

```

fill="none" stroke-linecap="round"/></svg>
      Retry
    </button>
  `;
  banner.classList.add("is-error");
  banner.hidden = false;
}
}

/**
 * Handle API and fetch errors
 */
function handleError(error) {
  console.error("App error:", error);

  let title = "Error Loading Charts";
  let message = "Something went wrong.
Please try again.";

  if (error.type === 'INVALID_API_KEY') {
    title = "YouTube API Key Missing or
Invalid";
    message = error.message;
  } else if (error.type ===
'QUOTA_EXCEEDED') {
    title = "Daily Quota Exceeded";
    message = error.message;
  } else if (error.type ===
'NETWORK_ERROR') {
    title = "Network Error";
    message = "Could not reach YouTube.
Check your internet connection.";
  } else if (error.type ===
'NO_SONGS_FETCHED') {

```

```

        title = error.type ===
'NO_SONGS_FETCHED' ? "No Songs Found" :
title;
        message = error.message;
    }

    showError(title, message);
}

/**
 * Render a song card from template
 */
function renderSongCard(song) {
    const template =
document.getElementById("song-card-
template");
    const clone =
template.content.cloneNode(true);

    const card =
clone.querySelector(".song-card");
    const thumb =
clone.querySelector(".thumb-img");
    const rankBadge =
clone.querySelector(".rank-badge");
    const duration =
clone.querySelector(".duration-badge");
    const playLink =
clone.querySelector(".play-overlay");
    const title =
clone.querySelector(".song-title");
    const artist =
clone.querySelector(".song-artist");
    const label =

```

```
clone.querySelector(".song-label");
    const uploadDate =
clone.querySelector(".song-date");
    const viewsEl =
clone.querySelector(".stat-views b");
    const likesEl =
clone.querySelector(".stat-likes b");
    const watchBtn =
clone.querySelector(".btn-watch");

    // Set image with lazy loading
    thumb.src = song.thumbnail ||
"data:image/svg+xml,%3Csvg
xmlns='http://www.w3.org/2000/svg'
viewBox='0 0 320 180'%3E%3Crect
fill='%231a1a1a' width='320'
height='180'%3E%3C/svg%3E";
    thumb.alt = song.title;

    // Set rank if available
    if (song.rank) {
        rankBadge.textContent = song.rank;
    }

    // Set duration
    if (song.formattedDuration) {
        duration.textContent =
song.formattedDuration;
    } else {
        duration.style.display = "none";
    }

    // Set links
    playLink.href = song.url;
```



```

        watchBtn.href = song.url;

        // Set text
        title.textContent = song.title;
        artist.textContent = song.channelName;
        label.textContent = song.label ||
song.channelName;
        uploadDate.textContent =
window.HVC.Ranking.getRelativeTime(song.uploadedAt);

        // Set stats
        viewsEl.textContent =
window.HVC.Ranking.formatNumber(song.views)
;
        likesEl.textContent =
window.HVC.Ranking.formatNumber(song.likes)
;

        return clone;
    }

    /**
     * Render a chart row (Top 10 / Fastest
     Rising)
     */
    function renderChartRow(song) {
        const template =
document.getElementById("chart-row-
template");
        const clone =
template.content.cloneNode(true);

        const row =

```

```

clone.querySelector(".chart-row");
    const rankNum =
clone.querySelector(".rank-number");
    const rankDelta =
clone.querySelector(".rank-delta");
    const thumb =
clone.querySelector(".thumb-img");
    const title =
clone.querySelector(".song-title");
    const artist =
clone.querySelector(".song-artist");
    const label =
clone.querySelector(".song-label");
    const uploadDate =
clone.querySelector(".song-date");
    const viewsEl =
clone.querySelector(".stat-views b");
    const likesEl =
clone.querySelector(".stat-likes b");
    const scoreEl =
clone.querySelector(".stat-score b");
    const playLink =
clone.querySelector(".btn-watch");

    rankNum.textContent = song.rank || "-";

    if (song.rankDelta !== undefined) {
        if (song.rankDelta > 0) {
            rankDelta.textContent =
`↑${song.rankDelta}`;
            rankDelta.classList.add("is-up");
        } else if (song.rankDelta < 0) {
            rankDelta.textContent =
`↓${Math.abs(song.rankDelta)}`;

```

```

        rankDelta.classList.add("is-down");
    } else {
        rankDelta.textContent = "→";
        rankDelta.classList.add("is-new");
    }
} else {
    rankDelta.textContent = "⊕";
    rankDelta.classList.add("is-new");
}

thumb.src = song.thumbnail ||
"data:image/svg+xml,%3Csvg
xmlns='http://www.w3.org/2000/svg'
viewBox='0 0 320 180'%3E%3Crect
fill='%231a1a1a' width='320'
height='180'/%3E%3C/svg%3E";
thumb.alt = song.title;

title.textContent = song.title;
artist.textContent = song.channelName;
label.textContent = song.label ||
song.channelName;
uploadDate.textContent =
window.HVC.Ranking.getRelativeTime(song.upl
oadedAt);

viewsEl.textContent =
window.HVC.Ranking.formatNumber(song.views)
;
likesEl.textContent =
window.HVC.Ranking.formatNumber(song.likes)
;
scoreEl.textContent =
Math.round(song.score.total);

```

```
        playLink.href = song.url;

        return clone;
    }

    /**
     * Render an artist card
     */
    function renderArtistCard(artist) {
        const template =
document.getElementById("artist-card-
template");
        const clone =
template.content.cloneNode(true);

        const name =
clone.querySelector(".artist-name");
        const avatar =
clone.querySelector(".artist-avatar img");
        const songCount =
clone.querySelector(".artist-song-count");
        const views =
clone.querySelector(".artist-views");

        name.textContent = artist.name;

        // Use the first song's channel as a
proxy for artist avatar
        if (artist.songs[0]?.thumbnail) {
            avatar.src =
artist.songs[0].thumbnail;
            avatar.alt = artist.name;
        }
    }
}
```

```

        songCount.textContent =
artist.songs.length;
        views.innerHTML =
`<b>${window.HVC.Ranking.formatNumber(artis
t.totalViews)}</b> total views`;

        return clone;
    }

    /**
     * Render trending ticker with top songs
     */
    function renderTrendingTicker() {
        const track =
document.getElementById("ticker-track");
        if (!track) return;

        const topSongs = rankedSongs.slice(0,
10);
        track.innerHTML = "";

        // Render each song twice for seamless
loop
        for (let i = 0; i < 2; i++) {
            topSongs.forEach(song => {
                const item =
document.createElement("span");
                item.className = "ticker-item";
                item.innerHTML = `<b>#${song.rank}
</b> ${escapeHtml(song.title.substring(0,
50))}`;
                track.appendChild(item);
            });
        }
    }

```

```

    }
  }

  /**
   * Render all homepage sections
   */
  function renderHomepage() {
    if (!document.getElementById("top10-
list")) return; // Not on homepage

    const banner =
document.getElementById("state-banner");
    if (rankedSongs.length === 0) {
      if (banner) {
        banner.classList.remove("is-
error");
        banner.innerHTML = `
          <h3>No Songs Yet</h3>
          <p>Add your first song in the
admin dashboard (/admin) and it will appear
here automatically – no page reload needed.
</p>
        `;
        banner.hidden = false;
      }
      return;
    } else if (banner) {
      banner.hidden = true;
    }

    // Filter by selected time range
    const filtered =
window.HVC.Ranking.filterByDateRange(ranked
Songs, currentFilter);

```

```
// Render Top 10
const top10 = filtered.slice(0,
window.HVC.CONFIG.SECTION_SIZES.top10);
renderList("top10-list", top10,
renderChartRow);

// Render New Releases
const newReleases =
window.HVC.Ranking.getNewReleases(filtered,
window.HVC.CONFIG.NEW_RELEASE_WINDOW_DAYS);
renderGrid("new-releases-grid",
newReleases.slice(0,
window.HVC.CONFIG.SECTION_SIZES.newReleases
), renderSongCard);

// Render Top Artists
const topArtists =
window.HVC.Ranking.getTopArtists(filtered,
window.HVC.CONFIG.TOP_CHART_SIZE);
renderArtistRow("top-artists-row",
topArtists.slice(0,
window.HVC.CONFIG.SECTION_SIZES.topArtists)
, renderArtistCard);

// Render Fastest Rising
const fastestRising =
window.HVC.Ranking.getFastestRising(filtere
d, window.HVC.CONFIG.RISING_WINDOW_DAYS);
renderList("fastest-rising-list",
fastestRising.slice(0,
window.HVC.CONFIG.SECTION_SIZES.fastestRisi
ng), renderChartRow);
```

```

        // Render Most Viewed
        const mostViewed =
window.HVC.Ranking.getMostViewed(filtered,
window.HVC.CONFIG.TOP_CHART_SIZE);
        renderGrid("most-viewed-grid",
mostViewed.slice(0,
window.HVC.CONFIG.SECTION_SIZES.mostViewed)
, renderSongCard);

        // Render Recently Added
        const recentlyAdded =
window.HVC.Ranking.getRecentlyAdded(filtere
d,
window.HVC.CONFIG.SECTION_SIZES.recentlyAdd
ed);
        renderGrid("recently-added-grid",
recentlyAdded, renderSongCard);

        // Render trending ticker
        renderTrendingTicker();
    }

    /**
     * Render a list of items (chart rows)
with staggered animation
     */
    function renderList(containerId, items,
renderFn) {
        const container =
document.getElementById(containerId);
        if (!container) return;

        container.innerHTML = "";
        items.forEach((item, index) => {

```



```

        const element = renderFn(item);
        // Stagger animation
        const wrapper =
document.createElement("div");
        wrapper.style.animationDelay =
`${index * 0.08}s`;
        wrapper.appendChild(element);
        container.appendChild(wrapper);
    });
}

/**
 * Render a grid of items (song cards)
with staggered animation
 */
function renderGrid(containerId, items,
renderFn) {
    const container =
document.getElementById(containerId);
    if (!container) return;

    container.innerHTML = "";
    items.forEach((item, index) => {
        const element = renderFn(item);
        // Stagger animation
        const wrapper =
document.createElement("div");
        wrapper.style.animationDelay =
`${index * 0.08}s`;
        wrapper.appendChild(element);
        container.appendChild(wrapper);
    });
}

```

```

/**
 * Render a row of artist cards with
staggered animation
 */
function renderArtistRow(containerId,
items, renderFn) {
    const container =
document.getElementById(containerId);
    if (!container) return;

    container.innerHTML = "";
    items.forEach((item, index) => {
        const element = renderFn(item);
        const wrapper =
document.createElement("div");
        wrapper.style.animationDelay =
`${index * 0.08}s`;
        wrapper.appendChild(element);
        container.appendChild(wrapper);
    });
}

/**
 * Attach event listeners to interactive
elements
 */
function attachEventListeners() {
    // Theme toggle
    const themeBtn =
document.getElementById("theme-toggle");
    if (themeBtn) {
        themeBtn.addEventListener("click",
toggleTheme);
        // Load saved theme

```

```
const savedTheme =
localStorage.getItem("hvc-theme");
if (savedTheme === "light") {

document.documentElement.setAttribute("data-
-theme", "light");
}
}

// Mobile nav toggle
const navToggle =
document.getElementById("nav-toggle");
const mainNav =
document.getElementById("main-nav");
if (navToggle && mainNav) {
  navToggle.addEventListener("click",
() => {
    mainNav.classList.toggle("is-
open");
    navToggle.setAttribute("aria-
expanded", mainNav.classList.contains("is-
open"));
  });
  // Close on nav link click
  mainNav.querySelectorAll(".nav-
link").forEach(link => {
    link.addEventListener("click", ()
=> {
      mainNav.classList.remove("is-
open");
      navToggle.setAttribute("aria-
expanded", "false");
    });
  });
}
```

```

    }

    // Filter chips
    const chips =
document.querySelectorAll(".chip");
    chips.forEach(chip => {
        chip.addEventListener("click", (e) =>
{
            chips.forEach(c =>
c.classList.remove("is-active"));
            e.target.classList.add("is-
active");
            currentFilter =
e.target.dataset.range;
            renderHomepage();
        });
    });

    // Search
    const searchForm =
document.getElementById("search-form");
    const searchInput =
document.getElementById("search-input");
    if (searchForm && searchInput) {
        searchForm.addEventListener("submit",
(e) => {
            e.preventDefault();
            const query =
searchInput.value.trim();
            if (query) {
                navigateToSearch(query);
            }
        });
    }
}

```

```
// Scroll to top
const scrollTopBtn =
document.getElementById("scroll-top-btn");
if (scrollTopBtn) {
  window.addEventListener("scroll", ()
=> {
    if (window.scrollY > 600) {
      scrollTopBtn.hidden = false;
    } else {
      scrollTopBtn.hidden = true;
    }
  });

scrollTopBtn.addEventListener("click", ()
=> {
  window.scrollTo({ top: 0, behavior:
"smooth" });
});
}

// Share button
const shareBtn =
document.getElementById("share-btn");
if (shareBtn) {
  shareBtn.addEventListener("click",
share);
}

// Copy link button
const copyBtn =
document.getElementById("copy-link-btn");
if (copyBtn) {
  copyBtn.addEventListener("click",
```

```

copyLink);
    }

    // Set footer year
    const yearEl =
document.getElementById("footer-year");
    if (yearEl) {
        yearEl.textContent = new
Date().getFullYear();
    }
}

/**
 * Toggle light/dark theme
 */
function toggleTheme() {
    const html = document.documentElement;
    const currentTheme =
html.getAttribute("data-theme");
    const newTheme = currentTheme ===
"light" ? "dark" : "light";

    html.setAttribute("data-theme",
newTheme);
    localStorage.setItem("hvc-theme",
newTheme);
}

/**
 * Navigate to search results
 */
function navigateToSearch(query) {
    const results =
window.HVC.Ranking.search(rankedSongs,

```

```
query));

    // Store in sessionStorage for search
    page to access
    sessionStorage.setItem("hvc-search-
    query", query);
    sessionStorage.setItem("hvc-search-
    results", JSON.stringify(results));

    // Redirect to search page (can be
    created separately or handle here)
    alert(`Found ${results.length} results
    for "${query}"`);
    }

    /**
    * Share the page
    */
    function share() {
        if (navigator.share) {
            navigator.share({
                title: "Haryanvi Charts",
                text: "Check out the latest
                Haryanvi music rankings!",
                url: window.location.href
            }).catch(err => console.log("Share
            failed:", err));
        } else {
            showToast("Share not supported on
            your device");
        }
    }

    /**
```

```

    * Copy page link to clipboard
    */
    function copyLink() {
        const url = window.location.href;

        navigator.clipboard.writeText(url).then(()
=> {
            showToast("Link copied to
clipboard!");
        }).catch(() => {
            showToast("Failed to copy link");
        });
    }

    /**
    * Show a toast notification
    */
    function showToast(message) {
        const toast =
document.getElementById("toast");
        if (!toast) return;

        toast.textContent = message;
        toast.classList.add("is-visible");

        setTimeout(() => {
            toast.classList.remove("is-visible");
        }, 3000);
    }

    /**
    * Escape HTML to prevent XSS
    */
    function escapeHtml(text) {

```



```
    const div =
document.createElement("div");
    div.textContent = text;
    return div.innerHTML;
  }

  // Initialize on DOM ready
  if (document.readyState === "loading") {

document.addEventListener("DOMContentLoaded
", init);
  } else {
    init();
  }

  // Public API for global access
  return {
    init,
    toggleTheme,
    share,
    copyLink,
    showToast,
    navigateToSearch
  };
})();
```